COMP 511

Computer Programming Foundations II

Stacks and Queues

Stack and Queue

Stack and Queues are linear data structures that follow a particular order to add or remove entities

Stack is LIFO (Last In First Out) while Queue is FIFO (First In First Out)

Both stacks and queues can also be implemented as array OR as a LinkedList

Stack Operations

- void push (x) ➔ Insert x
- void pop ( ) ➔ Remove most recently inserted item
- AnyType top ( ) ➔ Return most recently inserted item
- AnyType topAndPop ( ) ➔ Return and Remove most recently inserted item
- boolean isEmpty ( ) ➔ Return true if empty; else false
- void makeEmpty ( ) ➔ Remove all items

Queue Operations

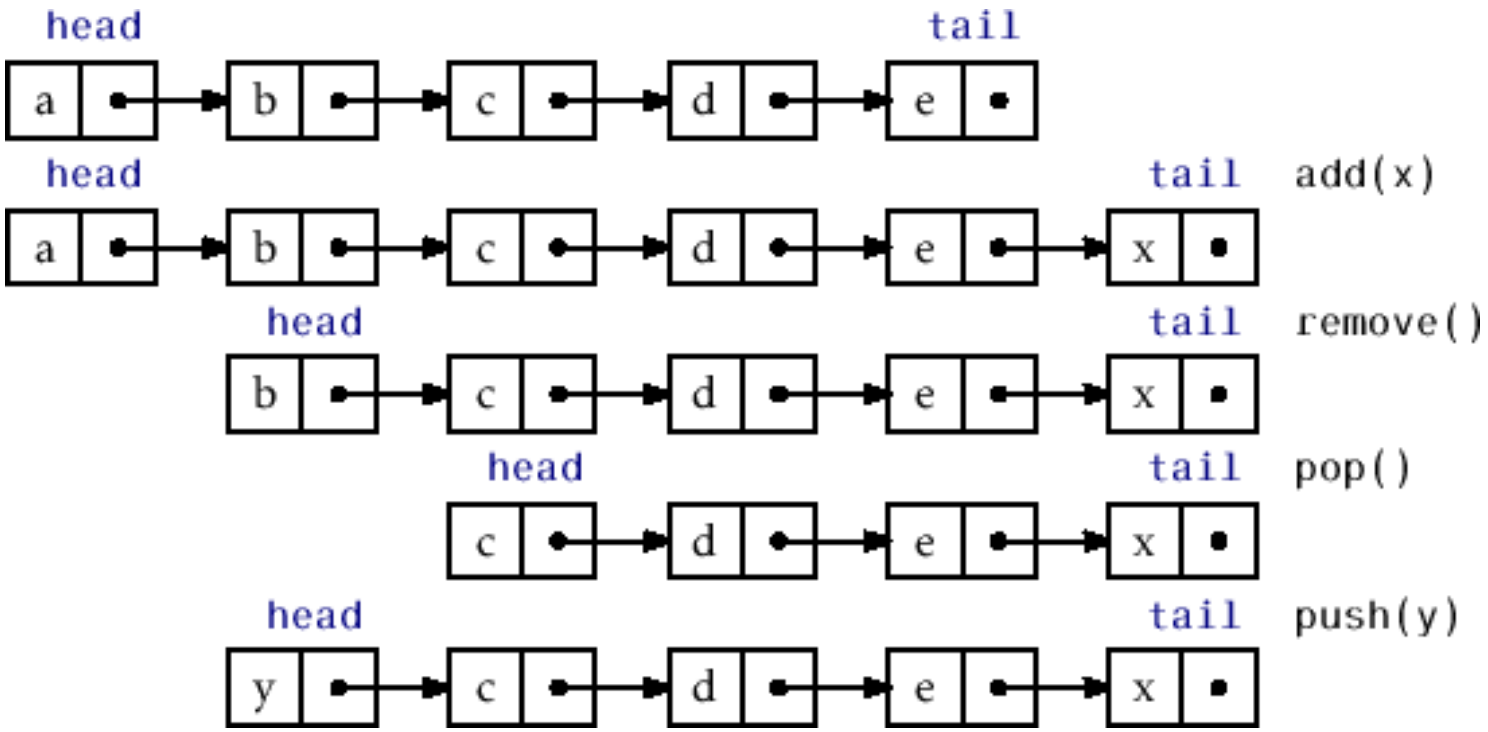
- void enqueue (x) ➔ Insert x
- AnyType getFront ( ) ➔ Return least recently inserted item
- AnyType dequeue ( ) ➔ Return and Remove least recently inserted item
- boolean isEmpty ( ) ➔ Return true if empty; else false
- void makeEmpty ( ) ➔ Remove all items

QueueArrayBased.java

```java
public class mainQueue {

    // main function
    public static void main (String[] args)
    {
        // create a queue of capacity 5
        QueueArrayBased q = new QueueArrayBased(5);

        q.enqueue(1);
        q.enqueue(2);
        q.enqueue(3);

        System.out.println("Front element is: " + q.peek());
        q.dequeue();
        System.out.println("Front element is: " + q.peek());
        System.out.println("Queue size is " + q.size());

        q.dequeue();
        q.dequeue();
        if (q.isEmpty())
            System.out.println("Queue Is Empty");
        else
            System.out.println("Queue Is Not Empty");
    }
}
```

```java
public class QueueArrayBased {
    private int arr[];      // array to store queue elements
    private int front;      // front points to front element in the queue (if any)
    private int rear;       // rear points to last element in the queue
    private int capacity;   // maximum capacity of the queue
    private int count;  // current size of the queue

    // Constructor to initialize queue
    QueueArrayBased(int size)
    {
        arr = new int[size];
        capacity = size;
        front = 0;
        rear = -1;
        count = 0;
    }
    // Utility function to remove front element from the queue
    public void dequeue()
    {
        // check for queue underflow
        if (isEmpty())
        {
            System.out.println("UnderFlow\nProgram Terminated");
            System.exit(1);
        }
        System.out.println("Removing " + arr[front]);
        front = (front + 1) % capacity;
        count--;
    }

    // Utility function to add an item to the queue
    public void enqueue(int item)
    {
        // check for queue overflow
        if (isFull())
        {
            System.out.println("OverFlow\nProgram Terminated");
            System.exit(1);
        }
        System.out.println("Inserting " + item);
        rear = (rear + 1) % capacity;
        arr[rear] = item;
        count++;
    }
```

Queue LinkedList Based



Queue LinkedList Based

queue: [10, 12, 14, 16, 18]

removed element-10

queue: [12, 14, 16, 18]

head:12

Size:4

```java
import java.util.LinkedList;
import java.util.Queue;


public class queueLinkedListBased {
    public static void main(String[] args)
    {
        Queue<Integer> queue = new LinkedList<>();

        for (int i=10; i<20; i+=2)
            queue.add(i);

        System.out.println("queue: "+queue);

        // Remove head of the queue.
        int headnode = queue.remove();
        System.out.println("removed element-" + headnode);

        System.out.println("queue: "+queue);

        // View head of the queue
        headnode = queue.peek();
        System.out.println("head:" + headnode);

        int size = queue.size();
        System.out.println("Size:" + size);
    }

}
```

Exercise

Download the File Queue.zip from the BlackBoard

Add a java class that does the following (as a simple main):

- Create a queue of capacity 5 (The same as shown in mainQueue class)
- enqueue three numbers then dequeue three times
- enqueue another three different numbers then dequeue once
- enqueue another three different numbers then dequeue two times
- enqueue another two different numbers then dequeue once
- Print the peek, the size and the content of the array

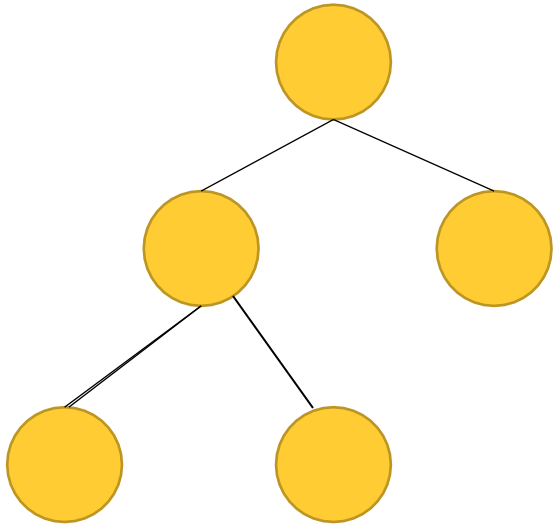
COMP 511

Computer Programming Foundations II

Binary Search Tree

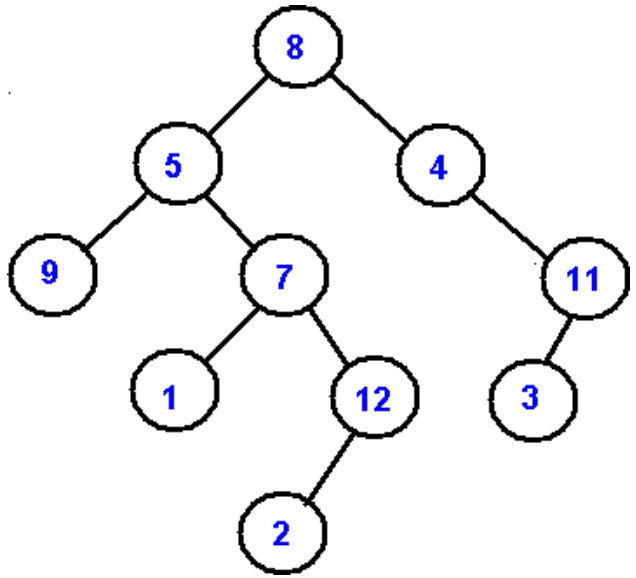
Binary Search Tree

- **Binary tree** is a tree in which no node can have more than two children.
- Extend the concept of linked data structures to structures containing nodes with more than one self-referenced field.
- Every node (excluding a root) in a tree is connected by a directed edge *from* exactly one other node. This node is called a **parent**.



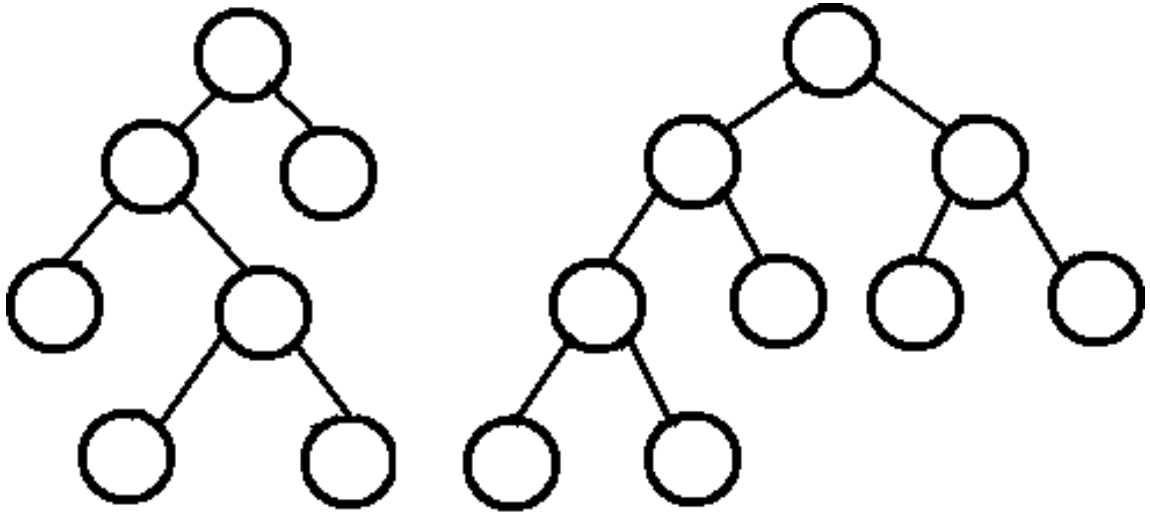
Binary Search Tree

- Each node connects *to* (at most) two nodes, called **children**
- Nodes with no children are called **leaves**
- Nodes which are not leaves are called **internal nodes**
- Nodes with the same parent are called **siblings**
- The **depth** of a node is the number of edges from the root to the node.
- The **height** of a node is the number of edges from the node to its deepest leaf.



Binary Search Tree

- A **full binary tree** is a binary tree in which each node has exactly zero or two children.
- A **complete binary tree** is a binary tree which is completely filled, with the possible exception of the bottom level, which is filled from left to right.



full tree complete tree

Search Tree Advantages

- Reflect structural relationships in the data
- Used to represent hierarchies
- Provide efficient insertion/deletion and searching mechanisms
- Flexible, allowing subtrees to be moved around with minimal effort

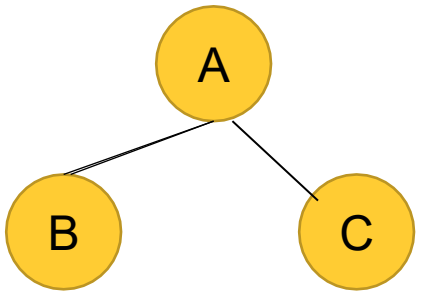
Binary Search Tree Traversal

- It is a process that visits all the nodes in the tree.
- Since a tree is a nonlinear data structure, there is no unique traversal

Traversal Algorithms

1. Depth-first traversal
2. Breadth-first traversal

Depth-First Traversal



PreOrder Traversal

Visit the parent first and then left and right children (PLR) A B C

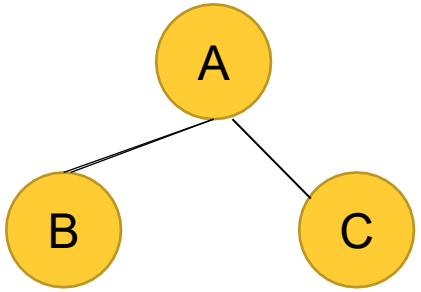
InOrder Traversal

Visit the left child, then the parent and the right child (LPR) B A C

PostOrder Traversal

Visit left child, then the right child and then the parent (LRP) B C A

Breadth-First Traversal



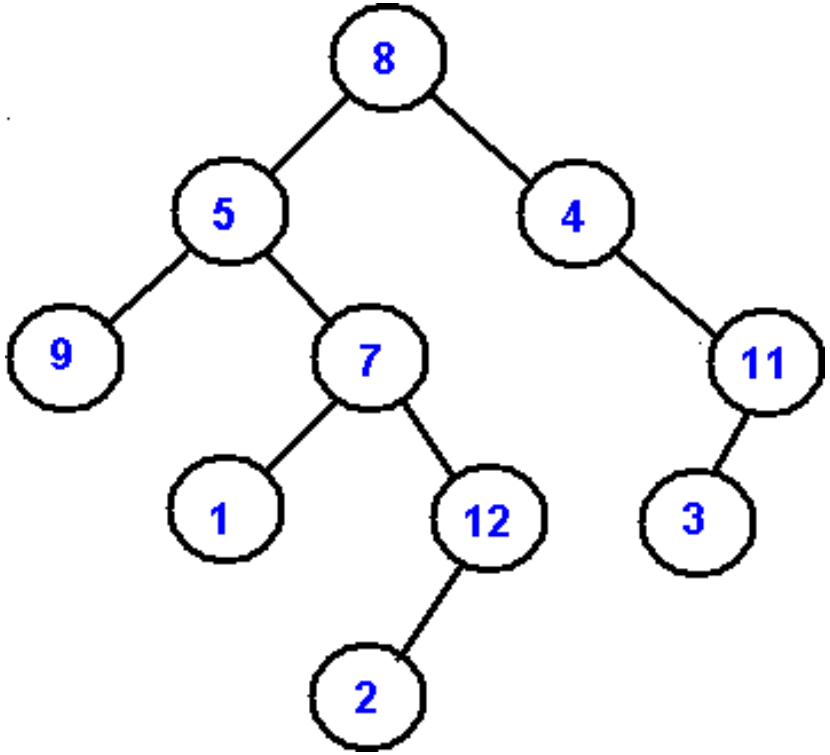
LevelOrder Traversal

This traversal visits nodes by levels from top to bottom and from left to right.

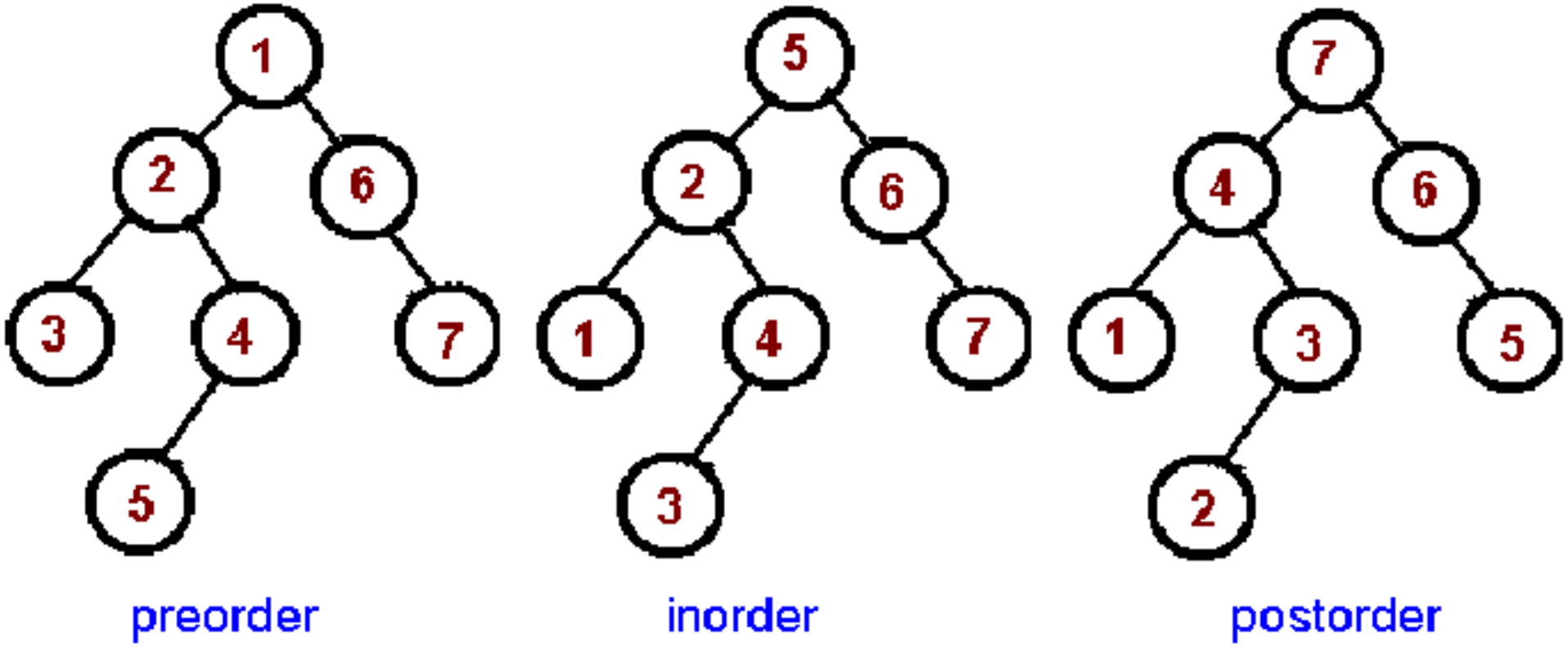
A B C

In-Class Traversal Example

PreOrder (PLR)	8, 5, 9, 7, 1, 12, 2, 4, 11, 3
InOrder (LPR)	9, 5, 1, 7, 2, 12, 8, 4, 3, 11
PostOrder (LRP)	9, 1, 2, 12, 7, 5, 3, 11, 4, 8
LevelOrder	8, 5, 4, 9, 7, 11, 1, 12, 3, 2



Binary Search Tree Traversal



```java
    //Preorder
    public void preorderPrint(BinaryTree bt) {
        if (bt == null) return;
        System.out.println(bt.value);
        preorderPrint(bt.leftChild);
        preorderPrint(bt.rightChild);
    }

    //Inorder
    public void inorderPrint(BinaryTree bt) {
        if (bt == null) return;
        inorderPrint(bt.leftChild);
        System.out.println(bt.value);
        inorderPrint(bt.rightChild);
    }


    //Postorder
    public void postorderPrint(BinaryTree bt) {
        if (bt == null) return;
        postorderPrint(bt.leftChild);
        postorderPrint(bt.rightChild);
        System.out.println(bt.value);
    }
}
```

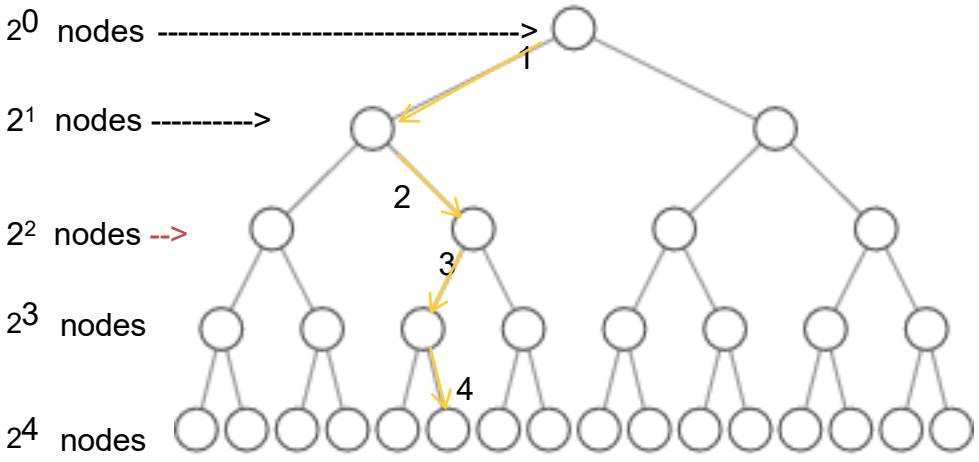
Binary Search Tree Traversal Basic Implementation

Logarithms

- $Log_2(n)$ provides us with the number of bits needed to represent n possibilities
    - To represent 10 digits, we need $log_2(10) = 3.322$ (round up to 4) bits
- Logarithms also tell us how many times we need to divide a positive integer in half before reaching 1
    - let's take the number 32 : $Log_2(32) = 5$

Binary Tree Complexity

Number of Nodes in Perfect Binary Tree

$$2^{h+1} - 1$$

Perfect Tree